CHAPTER 3
System Design and Architecture

3.1 Introduction

A clinical alert system cannot tolerate architectural ambiguity. The structural design must seamlessly synthesize electronic circuit topographies, multi-layered network routing strategies, and strict relational data models. In a critical care environment, system failure is not an acceptable failure mode; therefore, the structural design must incorporate explicit hardware- and software-level redundancies.

This chapter presents the comprehensive architectural blueprints of the proposed smart nurse call system. It breaks down the electronic circuit schematics for both the patient terminal and the central controller. It details the programmatic lifecycles driving the embedded firmware and provides an exhaustive analysis of the four-mode Adaptive Network Architecture. Finally, it outlines the non-volatile database schemas and communication protocols—including MQTT, WebSockets, and HTTPS—used to establish a secure and deterministic end-to-end communication pathway.

3.2 Global System Architecture

To resolve the dichotomy between local reliability and global cloud accessibility, the system is engineered around a hybrid, three-tier architecture, as illustrated in the system topology diagram.

1. The Device Layer (Patient Terminals): The foundational tier consists of multiple ESP8266 client nodes distributed across patient rooms. These low-power devices communicate exclusively via local Wi-Fi utilizing the lightweight MQTT protocol. Their sole responsibility is to monitor physical hardware interrupts and dispatch telemetry.

2. The Gateway Layer (Local Controller): The middle tier is occupied by the ESP32-S3 microcontroller. Acting as the localized brain of the ward, it hosts the embedded MQTT broker, captures all traffic from the patient rooms, triggers local physical alarms (such as acoustic buzzers), and serves an autonomous fallback web dashboard. It periodically bundles the system state for cloud synchronization.

3. The Cloud Platform (Backend & Dashboards): The top tier consists of a Node.js backend hosted on Railway and a React web dashboard deployed on Vercel. This layer provides global visibility, secure administration, and long-term data persistence via a PostgreSQL database. The local gateway synchronizes with this cloud platform via secure HTTPS requests at regular intervals.

3.3 Electronic Schematics and Pin Mapping

The physical circuitry of the system is intentionally minimalist. Reducing the number of discrete electronic components minimizes potential points of hardware failure while maximizing power efficiency and long-term stability.

3.3.1 Central Controller Configuration

The ESP32-S3 central controller is designed to provide immediate auditory warnings to medical staff present in the ward, operating independently of any web interface. A high-decibel active piezoelectric buzzer is connected directly to the controller's digital pins.

The positive terminal of the buzzer is mapped to General Purpose Input/Output (GPIO) pin 4, which is configured in software as a digital output. The negative terminal is routed to the common system ground (GND). When a medical alert payload is received via the MQTT broker, the controller drives GPIO 4 to a HIGH logic state (3.3V), continuously sounding the physical alarm until the alert is explicitly acknowledged and cleared by the nursing staff. The ESP32-S3 unit is powered continuously via its integrated USB-C port, drawing a standard 5V DC supply.

3.3.2 Patient Node Electronic Schematic

The physical circuit configuration of the portable patient node is engineered around the principle of strict power conservation and reliable edge-trigger execution. At the core of the schematic sits the ESP8266 (NodeMCU) microcontroller platform. The power subsystem is fed by a 3.7V nominal rechargeable lithium-ion cell, which serves as the primary DC energy reservoir. This arrangement is detailed in the circuit schematic in Figure 3.1.

[📸 DRAG AND DROP IMAGE HERE: Patient Node Circuit Schematic]

Figure 3.1: Electronic schematic of the patient node illustrating the wiring between the ESP8266, TP4056 charging module, battery, and tactile push-button.

The GPIO pin is configured in the embedded software as INPUT_PULLUP, meaning the microcontroller's internal resistor holds the pin at a stable 3.3V HIGH state during normal resting operation. When the patient depresses the button, the circuit closes to ground, creating a clean falling edge transition to a 0V LOW state. The microcontroller is programmed to register this specific active-low trigger as an emergency event, instantly initiating the alert transmission loop.

3.3.3 Visual Status Indicators (LED States)

To provide immediate operational feedback to both the patient and the medical staff, the patient terminal utilizes the ESP8266's integrated LED (mapped to GPIO 2, which operates as an active-low output). The firmware dictates specific blink patterns to communicate the device's network state:

• Fast Blink (200ms interval): The device is actively attempting to establish a connection to the local Wi-Fi network or the MQTT broker.

• Slow Blink (500ms interval): A network connection has been successfully established, but the device is currently awaiting administrative approval and room assignment on the central dashboard.

• Solid Illumination: An active emergency alert is in progress and has not yet been cleared by the staff.

• LED Off: The device is fully connected, approved by the administrator, and in a stable resting state, conserving battery power.

3.4 Mechanical and Ergonomic Industrial Design

The physical form factor of the patient remote control directly influences clinical effectiveness. If a handheld device is overly complex, heavy, or difficult to manipulate, its technical performance becomes irrelevant to a patient in distress. The mechanical architecture of the client node utilizes a custom-engineered enclosure optimized for accessibility, as shown in Figure 3.2.

[📸 DRAG AND DROP IMAGE HERE: 3D CAD Render of Patient Remote]

Figure 3.2: 3D CAD render of the physical handheld remote, detailing the ergonomic finger grooves and the placement of the activation button.

The client node casing is designed around a vertical, highly contoured handheld form factor, engineered using parametric computer-aided design (CAD) software and realized via high-resolution additive manufacturing (3D printing) using durable, non-toxic Acrylonitrile Butadiene Styrene (ABS) polymer.

The structural geometry incorporates clear lateral anatomical finger grooves recessed into the left and right structural walls of the chassis. These grooves provide natural tactile anchoring points that conform to the geometry of a human fist. This design selection is vital for patients suffering from post-operative tremors, advanced muscular dystrophy, generalized neurological weakness, or arthritic joint degeneration, allowing them to retain a firm grip on the remote without requiring precise fine-motor control.

3.5 Software Architecture and Logic Flowcharts

The system's software architecture relies on asynchronous, non-blocking firmware routines designed to prevent code freezing and ensure real-time responsiveness across all operational nodes. The execution logic of the patient node is strictly defined to conserve power and guarantee transmission.

[📸 DRAG AND DROP IMAGE HERE: Firmware Execution Logic Flowchart]

Figure 3.3: Logic flowchart detailing the software execution loop of the patient node upon mechanical activation.

Initialization Phase: Upon cold boot or hardware wake, the processor executes a core setup routine that maps the internal clock speeds, configures the serial communication interfaces for diagnostic telemetry, and sets the designated trigger pin to an input state with internal pull-ups enabled. The hardware interrupt is attached to this pin, configured to watch for a falling edge transition.

Interrupt Event Handshake: When the patient button is depressed, standard script routines are bypassed as the CPU executes the interrupt loop. The firmware enters a localized hardware debouncing validation phase, monitoring the pin state for a sustained 50-millisecond window to screen out microscopic physical spring bounce artifacts.

Telemetry Dispatch: Once validated, the firmware establishes a secure socket link to the local MQTT broker port (1883). It constructs a compact JSON or raw string array payload embedding its hardcoded physical address and an active emergency status flag, immediately publishing the data packet into the designated network topic tree.

3.6 The Four-Mode Adaptive Network Architecture

Hospital infrastructures vary drastically—from modern urban facilities equipped with enterprise-grade Wi-Fi to isolated rural clinics lacking any pre-existing network infrastructure. To guarantee operational continuity across all deployment environments, the system features a highly versatile, four-mode Adaptive Network Architecture, selectable during the initial setup wizard. This architecture is mapped out in Figure 3.4.

[📸 DRAG AND DROP IMAGE HERE: 4-Mode Network Architecture Diagram]

Figure 3.4: Visual representation of the four routing modes (Standalone AP, Hospital Wi-Fi, Hybrid Bridge, and Cloud Sync) dictating the system's fail-safe logic.

3.6.1 Mode 1: Local Access Point (AP Only)

In Mode 1, the ESP32-S3 central hub completely isolates its radio frequency processors from external networks. It configures itself to operate as an independent software access point (SoftAP), broadcasting a localized Wi-Fi network named 'HospitalAlarm'.

Patient client nodes are configured to bind directly to this unique, localized network. All MQTT telemetry and HTTP dashboard traffic remain contained within this isolated airspace. The dashboard is accessed by navigating to the ESP32-S3's default local IP address (192.168.4.1). This mode requires zero external internet or existing hospital infrastructure, providing a fail-proof deployment model for isolated wards or temporary triage zones.

3.6.2 Mode 2: Local Station (STA Only)

In Mode 2, the ESP32-S3 acts as a standard wireless client, connecting directly to the healthcare facility’s existing enterprise-grade local area network (LAN). The remote patient nodes are configured to connect to the same building-wide access points.

All communications are routed through the facility's existing network switches. This mode allows the localized web dashboard to be securely accessed from any authorized computer terminal or tablet connected to the hospital's internal network, seamlessly integrating the alert architecture with the facility's existing IT infrastructure.

3.6.3 Mode 3: Redundant Hybrid (AP + STA)

Mode 3 represents a hybrid configuration designed to eliminate network single points of failure. The ESP32-S3's network processor concurrently executes both station (STA) and soft access point (SoftAP) protocols.

The central hub connects to the hospital's infrastructure Wi-Fi network to provide facility-wide dashboard routing, while simultaneously broadcasting its own private, localized backup wireless footprint ('HospitalAlarm'). Patient devices can failover between the two networks, ensuring that local alert routing continues uninterrupted even if the primary hospital network experiences a total collapse.

3.6.4 Mode 4: Cloud Synchronization (Online)

Operating functionally like Mode 3, Mode 4 activates the system's external IoT gateway protocols. A critical design feature of this architecture is the persistence of the MQTT protocol. While MQTT is utilized locally between the patient nodes and the central controller, it is also explicitly deployed for the global connection between the ESP32-S3 controller and the external cloud infrastructure.

This dual-layered MQTT synchronization ensures that telemetry remains ultra-lightweight and heavily resilient against latency, even over unstable wide-area networks. Mode 4 enables the web-based clinical dashboard to be securely monitored from any authorized device globally. It facilitates the use of the native Capacitor mobile application, pushing real-time notifications to nurses' smartphones, making it the optimal configuration for large-scale, modern medical networks.

3.7 System Security and Resilience

A medical system is fundamentally defined by its fault tolerance. The architecture incorporates multiple layers of automated resilience to proactively protect patient safety.

3.7.1 Local Isolation and Offline Continuity

The most critical vulnerability of modern IoT systems is cloud dependency. This project resolves this via its Mode 4 failover design. If the hospital's primary internet connection is severed, the ESP32-S3 gracefully degrades its operation. It abandons external synchronization but maintains the local MQTT broker and the embedded fallback web server.

Nurses on the floor can connect a tablet directly to the ESP32-S3's localized Wi-Fi (IP 192.168.4.1) to view and acknowledge live alerts without interruption. Once external internet is restored, the gateway automatically reconnects and silently back-syncs the state to the PostgreSQL database.

3.7.2 Connection Monitoring (Heartbeat)

Wireless networks are inherently susceptible to interference. To prevent a scenario where a nurse falsely believes a patient is safe when their device has actually lost power, the system implements strict heartbeat timeouts.

Every ESP8266 node emits a background ping to the broker. If the ESP32-S3 does not register a heartbeat from a node for 30 consecutive seconds, it actively alters the device's state to offline. This instantly darkens the patient's status card on the web dashboard, providing immediate visual warning of a potential hardware anomaly.